

Zero to Hero Study Handbook: Production Internet AI Agent

This handbook is based on static analysis of the repository files in this project. It is intended for self-study and code understanding, not runtime validation.

Module 1: Foundations & Architecture

1) What this project does

This project implements a production-style internet-connected AI agent platform with four user surfaces:

- FastAPI service in `src/internet_agent/api/app.py`
- Typer CLI in `src/internet_agent/cli.py`
- Streamlit dashboard in `streamlit_app/`
- MCP-compatible stdio server in `src/internet_agent/mcp/server.py`

The core runtime (`InternetAgentService` in `src/internet_agent/services/agent_service.py`) orchestrates:

- LangGraph multi-agent workflow (`InternetAgentWorkflow`)
- Hybrid retrieval (search + web fetch/extract + chunk + semantic memory)
- Persistent memory in SQLite and vector memory in ChromaDB
- Report generation (markdown, html, json, pdf)
- Monitoring and analytics

Main use cases:

- Time-sensitive Q&A that needs web evidence (`latest`, `today`, `news`)
- Local reasoning using semantic memory for non-time-sensitive questions
- Tool-augmented tasks (weather, currency, calculator, file readers, etc.)
- Exportable run reports for traceability

2) Core paradigms and patterns used here

Definitions first, then where they appear:

- State-machine workflow: A process is modeled as named states/nodes with transitions. Used in `src/internet_agent/agent/workflow.py` with LangGraph nodes like `user_intent_agent`, `planner_agent`, `search_agent`, `verification_agent`.
- Service orchestration: A single service object coordinates many subsystems. Used by `InternetAgentService` in `src/internet_agent/services/agent_service.py`.
- Repository pattern: Data access is centralized behind a repository interface. Used by `MemoryRepository` in `src/internet_agent/memory/repository.py`.

- Schema-first contracts: Request/response payloads are explicitly typed with Pydantic models. Used in `src/internet_agent/api/schemas.py` and tool input/output models in `src/internet_agent/tools/*`.
- Async I/O: Network-bound operations are `async` to avoid blocking. Used in `fetch/search/tool` execution and FastAPI endpoints.
- Plugin extension pattern: Tool factories can be injected by dotted-path strings. Implemented in `src/internet_agent/tools/plugins.py`.
- Observability instrumentation: Structured logging + counters + latency histograms. Implemented in `src/internet_agent/logging_utils.py` and `src/internet_agent/metrics.py`.

3) Architecture and interactions

Top-level runtime composition is defined in `InternetAgentService.__init__`:

- Config: `get_settings()`
- Logging: `configure_logging(...)`
- Memory: `MemoryRepository`, `ChromaMemoryStore`
- Tools: `build_default_registry(...)` (+ optional plugins)
- Models: `OllamaClient`
- Workflow: `InternetAgentWorkflow`
- Retrieval: `RetrievalPipeline`
- Fetcher: `WebsiteFetcher`
- Reporting: `ReportService`

Text-based main flow:

```

User (CLI/API/Streamlit/MCP)
  |
  v
InternetAgentService
  |
  v
InternetAgentWorkflow (AgentState)
  user_intent_agent
    -> planner_agent
    -> search_decision_agent
        -> (need_internet = true) search_agent -> web_extraction_agent
        -> summarization_agent
        -> verification_agent
            -> retry_search? (loop) or memory_agent
        -> reflection_agent
        -> report_agent
  |
  v
MemoryRepository (SQLite) + ChromaMemoryStore (vector memory)

```

|
v

Response payload + optional report artifacts

Retrieval sub-flow in `src/internet_agent/retrieval/pipeline.py`:

```

query
-> cache_get(search:<sha256>)
-> semantic_store.query(...)
-> SearchProviders.search_all(...)
-> rank_sources(...)
-> WebsiteFetcher.fetch(...) for top URLs
-> clean_html/extract_markdown/read_pdf_bytes
-> chunk_text(...)
-> semantic_store.upsert(...)
-> cache_set(...)
-> return results/documents/chunks/latency

```

Module 2: Repository Map

Core files a new contributor should study first:

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
<code>pyproject.toml</code>	Packaging, dependencies, CLI/API/MCP entry points	<code>[project.scripts]</code> -> <code>internet-agent</code> , <code>internet-agent-api</code> , <code>internet-agent-mcp</code>	<code>requires-python = >=3.12,<3.13</code>
<code>configs/config.yaml</code>	Runtime defaults	N/A (data file)	<code>llm.*</code> , <code>agent.*</code> , <code>search.*</code> , <code>retrieval.*</code> , <code>memory.*</code> , <code>cache.*</code> , <code>api.*</code> , <code>reports.*</code>
<code>src/internet_agent/config.py</code>	Configuration loading and env overrides	<code>Settings</code> , <code>get_settings</code> , <code>_env_overrides</code> , <code>ensure_runtime_dirs</code>	Env prefix <code>INTERNET_AGENT__</code>
<code>src/internet_agent/infrastructure/agent_orchestrator.py</code>	Orchestration for all frontends	<code>InternetAgentService</code>	<code>self.chat/search_enabled</code> , <code>self.history/memory_search</code> , <code>settings.monitoring.*</code>
<code>src/internet_agent/agent/state.py</code>	Agent state schema	<code>AgentState</code> , <code>can_retry_search</code>	<code>verification_loops</code> , <code>retry_search</code> , <code>confidence</code> , <code>reasoning_trace</code>

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/internet_agent/agent/workflow.py	Agent workflow and control flow	InternetAgentWorkflow, all *_agent methods, _route_after_*	settings.agent.max_verification_loops, verification_confidence_threshold
src/internet_agent/ollama/client.py	Ollama client invocation and JSON parsing	OllamaClient.ask, ask_json, _is_ollama_available	settings.llm.base_url, temperature, max_tokens, request_timeout_seconds
src/internet_agent/external/retrieval/pipeline.py	External retrieval pipeline	ExternalRetrievalPipeline, _fetch_and_extract, _chunk_documents	retrieval.chunk_size, chunk_overlap, max_urls_per_query, max_content_chars
src/internet_agent/external/retrieval/providers.py	External retrieval providers fan-out	SearchProviders, search_duckduckgo, search_wikipedia, search_git	search_providers, search_hybrid_policy
src/internet_agent/external/retrieval/providers/fetch.py	External retrieval providers fetch retry	WebsiteFetcher.fetch, (tenacity), timeout	
src/internet_agent/external/retrieval/providers/extract.py	External retrieval providers extraction	extract_html, extract_markdown, read_pdf_bytes	Uses traifatura, BeautifulSoup, pypdf
src/internet_agent/external/retrieval/providers/ranker.py	External retrieval providers ranking	_rank_sources, _authority_boost, _freshness_score	rank_score formula (relevance + authority + freshness)
src/internet_agent/memory/models.py	Memory models schema	ConversationMessage, SearchRecord, VisitedURL, RetrievedDocument, SummaryRecord, ToolHistory, ReportRecord, CacheEntry	Table names and JSON columns
src/internet_agent/memory/repository.py	Memory repository persistence API	add_message, get_messages, cache_get, cache_set, etc.	make_cache_key(namespace, payload)
src/internet_agent/memory/chroma.py	Memory chroma memory	ChromaMemoryStore, query	memory.chroma_path, memory.chroma_collection
src/internet_agent/tools/base.py	Tools base contract	BaseTool, ToolDescriptor	input_model, output_model

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
src/internet_agent/Tools/registry.py	ToolRegistry + invocation telemetry	ToolRegistry.register	vars/discover/invoke history and metrics
src/internet_agent/Tools/factory.py	ToolFactory composition	build_default_registry	Registers search/utility/local tools
src/internet_agent/Tools/plugins.py	ToolPlugin loading	ToolPluginLoader	PluginManager, registry
src/internet_agent/Api/schemas.py	API contracts	ChatRequest/Response, SearchRequest/Response, BrowseRequest, ReportRequest, MemoryRequest	Response field
src/internet_agent/Api/app.py	API endpoints	/chat, /search, /browse, /history, /memory, /report, /health, /metrics, /monitor, /analytics	verify_api_key dependency on protected routes
src/internet_agent/Api/auth.py	API auth	verify_api_key	api.require_api_key, env var
src/internet_agent/CLI.py	CLI command surface	chat, search, summarize, report, memory, doctor	api.api_key_env default session_id = "cli"
src/internet_agent/MCP/Server.py	MCP/Server.py stdio server	MCPServer.handle_request, run_stdio_server	methods: initialize, tools/list, tools/call
streamlit_app/HomeDashboard UI + pages	Dashboard UI + pages	Page scripts call get_service() and service methods	Session state keys (session_id, last_chat, last_search)
streamlit_app/pages/*			
tests/test_api.py	API contract expectations (with fake service)	test_api_endpoint	Endpoint payload shape assumptions
tests/test_tools.py	Tool behavior examples	test_calculator_tool, test_unit_convert_tool	Calculator and unit conversion outputs

File/Directory Path	Primary Responsibility	Key Classes/Functions	Important Con-figs/Variables
<code>.env.example</code>	Optional env keys	N/A	INTERNET_AGENT_API_KEY, OLLAMA_BASE_URL, MLFLOW_TRACKING_URI, etc.

Module 3: Core Execution Flows

Flow A: FastAPI /chat request-to-response

Code path:

1. HTTP POST /chat enters `src/internet_agent/api/app.py::chat`.
2. Request body is parsed by `ChatRequest`:
 - `session_id: str` (default "default")
 - `message: str`
3. Endpoint calls:
 - `await get_service().chat(session_id=request.session_id, message=request.message)`
4. `InternetAgentService.chat(...)` runs:
 - `state = await self.workflow.run(...)`
 - builds response dict with fields:
 - `session_id`
 - `query`
 - `answer`
 - `confidence`
 - `hallucination_risk`
 - `citations`
 - `reasoning_trace`
 - `tool_outputs`
 - `report`
5. API endpoint casts payload into `ChatResponse` (without `report` because response model excludes it).

Expected API JSON shape (`ChatResponse`):

```
{
  "session_id": "string",
  "query": "string",
  "answer": "string",
  "confidence": 0.0,
  "hallucination_risk": "string",
  "citations": [],
  "reasoning_trace": [],
  "tool_outputs": []
}
```

Flow B: LangGraph multi-agent reasoning path

Primary function: `InternetAgentWorkflow.run(session_id, query)`.

Detailed sequence:

1. Build `AgentState(session_id, user_query)`.
2. Run `user_intent_agent`:
 - heuristic intent classification (`weather_lookup`, `currency_or_unit`, `time_sensitive_search`, `general_qa`)
3. Run `planner_agent`:
 - sets `plan_steps` and `planned_tools` (e.g., `weather/currency/calculator`)
4. Run `search_decision_agent`:
 - queries semantic cache first via `semantic_store.query(...)`
 - if cache is useful and query not time-sensitive, set `need_internet = False`
 - else calls `llm.ask_json(...)` with `prompts/search_planning.md`
5. If internet needed, loop:
 - `search_agent`
 - `web_extraction_agent`
 - `summarization_agent`
 - `verification_agent`
 - repeat if `state.can_retry_search(...)` is true
6. If internet not needed:
 - `summarization_agent` then `verification_agent`
7. Final stages:
 - `memory_agent`
 - `reflection_agent`
 - `report_agent`

Important state fields mutated across nodes:

- Planning: `intent`, `plan_steps`, `planned_tools`
- Retrieval: `need_internet`, `selected_providers`, `search_results`, `retrieved_documents`, `retrieved_chunks`
- Quality: `confidence`, `hallucination_risk`, `missing_info`, `conflicts`, `retry_search`, `verification_loops`
- Output: `draft_answer`, `final_answer`, `citations`, `report_payload`, `done`

Flow C: Retrieval pipeline (search operation)

Primary function: `RetrievalPipeline.run(session_id, query, providers=None)`.

Step-by-step:

1. Compute cache key via `MemoryRepository.make_cache_key("search", query)`.
2. If `cache.enabled`, try `memory_repo.cache_get(key)`.

3. Query semantic memory: `semantic_store.query(query, top_k=settings.memory.memory_top_k)`.
4. Provider fan-out: `SearchProviders.search_all(...)` returning:
 - `dict[str, list[dict]]` keyed by provider (duckduckgo, news, wikipedia, github)
5. Flatten and rank with `rank_sources(...)` -> adds `rank_score`.
6. Fetch and extract top URLs with `_fetch_and_extract(...)`:
 - `WebsiteFetcher.fetch(url)` returns raw payload (`url`, `status_code`, `content_type`, `text`, `bytes`)
 - HTML: `clean_html + extract_markdown`
 - PDF: `read_pdf_bytes`
7. Chunk documents with `_chunk_documents(...)` using `chunk_text(...)`.
8. Upsert chunks into Chroma with deterministic sha256 IDs.
9. Cache final result (if enabled).

Return payload shape:

```
{
  "query": "string",
  "providers": ["duckduckgo", "news"],
  "semantic_hits": [],
  "results": [],
  "documents": [],
  "chunks": [],
  "from_cache": false,
  "latency_ms": 0.0
}
```

`results` row fields include (from provider + ranker): `title`, `url`, `snippet`, `source`, `published`, optional `stars`, plus `rank_score`.

Flow D: Tool invocation and history logging

Core path: `ToolRegistry.invoke(session_id, name, payload)`.

1. Validate input using each tool's `input_model`.
2. Execute `await tool.run(validated_payload)`.
3. Validate output with `output_model`.
4. Record metrics:
 - `METRICS.inc(f"tool.{name}.success|error")`
 - `METRICS.observe_ms(f"tool.{name}.latency_ms", latency_ms)`
5. Persist tool history via `memory_repo.add_tool_history(...)`.

Stored tool history shape from `MemoryRepository.get_tool_history(...)`:

```
{
  "tool_name": "string",
  "input": {},
  "output": {},
  "status": "ok|error",
}
```



```

    "latency_ms": 0.0,
    "created_at": "ISO8601"
}

```

Flow E: Report generation and persistence

1. Export endpoint/CLI/Streamlit calls:
 - `InternetAgentService.export_report(session_id, payload, fmt)`
2. Delegates to `ReportService.generate(...)`.
3. Supported formats in code:
 - `markdown, html, json, pdf`
4. Generated file path is stored in SQLite via `memory_repo.add_report(...)`.

ReportRequest input shape:

```

{
  "session_id": "default",
  "format": "json",
  "payload": {}
}

```

Module 4: Setup & Run Guide

This section describes setup from repository manifests/configs, without executing code.

1) Prerequisites

From `pyproject.toml`:

- Python `>=3.12, <3.13`
- `uv` for environment and dependency management

Core runtime dependencies include:

- `langgraph, langchain, langchain-ollama`
- `fastapi, uvicorn, streamlit, typer`
- `chromadb, sentence-transformers`
- `sqlalchemy`
- `httpx, duckduckgo-search/ddgs, beautifulsoup4, trafilatura, pypdf`

2) Environment setup

Typical sequence (from repository conventions):

```

uv venv .venv
source .venv/bin/activate
uv sync --all-groups

```

3) Environment variables

From `.env.example` and config interpolation:

- `INTERNET_AGENT_API_KEY` (used when `api.require_api_key: true`)
- `OLLAMA_BASE_URL` (defaults to `http://127.0.0.1:11434`)
- `MLFLOW_TRACKING_URI` (optional override)
- Optional provider keys listed in `.env.example`:
 - `GITHUB_TOKEN`
 - `NEWS_API_KEY`
 - `OPENWEATHER_API_KEY`
 - `EXCHANGERATE_API_KEY`

Config also supports nested env overrides with prefix:

- `INTERNET_AGENT__<SECTION>__<KEY>=...`

Example:

```
INTERNET_AGENT__API__REQUIRE_API_KEY=true
```

4) Main configuration file

Primary config: `configs/config.yaml`.

Critical keys for first-time understanding:

- Model routing: `llm.planning_model`, `llm.reasoning_model`, `llm.summarization_model`, `llm.verification_model`, `llm.reflection_model`
- Workflow control: `agent.max_iterations`, `agent.max_verification_loops`, `agent.verification_confidence_threshold`
- Retrieval sizing: `retrieval.chunk_size`, `retrieval.chunk_overlap`, `retrieval.max_urls_per_query`, `retrieval.max_content_chars`
- Memory paths: `memory.sqlite_url`, `memory.chroma_path`, `memory.chroma_collection`
- API controls: `api.host`, `api.port`, `api.require_api_key`, `api.api_key_env`

5) Entry points and launch commands

From `[project.scripts]`:

```
uv run internet-agent
uv run internet-agent-api
uv run internet-agent-mcp
```

Typical direct launches from code layout:

```
uv run python app.py
uv run uvicorn internet_agent.api.app:app --host 0.0.0.0 --port 8000
uv run streamlit run streamlit_app/Home.py
uv run internet-agent doctor
```

6) Database/migration/seeding notes

There is no separate migration framework in this repo.

- SQLite schema is created automatically by `MemoryRepository.__init__`:
 - It initializes `Database(settings)` then runs `db.create_all()`.
- Chroma collection is initialized by `ChromaMemoryStore.__init__` with:
 - `chromadb.PersistentClient(path=settings.memory.chroma_path)`
 - `get_or_create_collection(name=settings.memory.chroma_collection, metadata={"hnsw:space": "cosine"})`

So first runtime initialization creates required local persistence structures automatically.

Module 5: Study Plan & Practice Exercises

A) Ordered study plan

Recommended reading order for new learners:

1. `pyproject.toml` and `configs/config.yaml`
 - Understand entry points, dependencies, and runtime knobs.
2. `src/internet_agent/config.py`
 - Learn how typed settings and env overrides work.
3. `src/internet_agent/services/agent_service.py`
 - See the full system composition in one place.
4. `src/internet_agent/agent/state.py` and `src/internet_agent/agent/workflow.py`
 - Learn the multi-agent orchestration lifecycle.
5. Retrieval package:
 - `retrieval/search.py`
 - `retrieval/pipeline.py`
 - `retrieval/extract.py`, `retrieval/chunking.py`, `retrieval/ranker.py`
6. Memory package:
 - `memory/models.py`
 - `memory/repository.py`
 - `memory/chroma_store.py`
7. Tools package:
 - `tools/base.py`, `tools/registry.py`, `tools/factory.py`
 - then `tools/search_tools.py`, `tools/utility_tools.py`, `tools/local_tools.py`
8. Interfaces:
 - `api/app.py` + `api/schemas.py`
 - `cli.py`
 - `streamlit_app/`
 - `mcp/server.py`
9. Tests:
 - `tests/test_api.py`, `tests/test_tools.py`, `tests/test_memory_cache.py`, `tests/test_config.py`

B) Practice exercises

1. Trace `/chat` end-to-end.
 - Task: Starting at `api/app.py::chat`, list every method called until final response is returned.
2. Explain internet decision logic.
 - Task: In `agent/workflow.py::search_decision_agent`, identify all conditions that set `need_internet`.
3. Inspect retry behavior.
 - Task: Find where verification can trigger an additional search and how max loops are enforced.
4. Map retrieval payload schema.
 - Task: In `retrieval/pipeline.py::run`, document each top-level key in the returned dict and where it comes from.
5. Review persistence model.
 - Task: In `memory/models.py`, match each ORM table to the method(s) writing to it in `memory/repository.py`.
6. Understand tool safety constraints.
 - Task: Compare `CalculatorTool` and `PythonCalculatorTool` in `tools/utility_tools.py`. What AST restrictions are applied?
7. Identify plugin insertion point.
 - Task: Show where plugin tools are loaded and how factory strings are interpreted.
8. Analyze report formats.
 - Task: In `services/report_service.py`, compare generated content shape for markdown vs json vs pdf.
9. Follow Streamlit state usage.
 - Task: Identify where `st.session_state["session_id"]`, `["last_chat"]`, and `["last_search"]` are written/read across pages.
10. Reconstruct API contract.
 - Task: Build request/response examples for `/search`, `/browse`, `/memory`, `/report` using `api/schemas.py` and endpoint code.

C) Solution outlines (brief)

1. `/chat` flow:
 - `api.chat` -> `InternetAgentService.chat` -> `InternetAgentWorkflow.run` -> agent nodes -> returned state mapped into `ChatResponse`.
2. `need_internet` logic:
 - False when semantic cache has content and query is not time-sensitive.
 - Otherwise inferred from LLM JSON (`search_planning.md`) with heuristic fallback for time-sensitive intent.
3. Retry behavior:
 - Set in `verification_agent` using `retry_search` from verification output and confidence threshold comparison.

- Loop enforced by `state.can_retry_search(max_verification_loops)`.
- Retrieval dict keys:
 - `query`, `providers`, `semantic_hits`, `results`, `documents`, `chunks`, `from_cache`, `latency_ms`.
 - Built from provider fan-out, ranking, extraction, chunking, and timers.
 - Table-to-method mapping examples:
 - `conversation_messages <- add_message`
 - `search_records <- add_search_record`
 - `retrieved_documents <- add_document`
 - `tool_history <- add_tool_history`
 - `report_records <- add_report`
 - `cache_entries <- cache_set`
 - Tool safety:
 - `CalculatorTool` allows only arithmetic node types in AST.
 - `PythonCalculatorTool` restricts imports and allows `math`-scoped expression evaluation.
 - Plugin loading:
 - In `InternetAgentService.__init__`, when `settings.plugins.tool_factories` is non-empty.
 - `ToolPluginLoader` parses `package.module:function_name`, calls the factory, validates instances as `BaseTool`.
 - Report format comparison:
 - Markdown: narrative sections (`Answer`, `Citations`, `Verification`, `Reasoning Trace`).
 - JSON: serialized payload dict.
 - PDF: flattened text lines using `FPDF.multi_cell`.
 - Streamlit state:
 - `2_Chat.py` writes `session_id` and `last_chat`.
 - `3_Search_Explorer.py` writes `last_search` and `session_id`.
 - `4_Retrieved_Documents.py` reads `last_search`.
 - `7_Reports.py` reads `last_chat`.
 - API contracts:
 - Directly from `api/schemas.py` models plus endpoint method signatures in `api/app.py`.

Learner Self-Verification Checklist

Use this checklist after studying:

- I can explain the role of `InternetAgentService` and name its core dependencies.
- I can trace `AgentState` field changes across the workflow nodes.
- I can explain when the system uses semantic cache vs live internet search.
- I can describe the full retrieval pipeline from provider results to Chroma upsert.
- I can map each persistent table to write/read methods in `MemoryRepository`.

- I can describe tool registration, invocation, validation, and telemetry logging.
- I can reconstruct FastAPI request/response schemas without guessing.
- I can explain how report generation differs across `markdown`, `html`, `json`, and `pdf`.
- I can explain how Streamlit pages share runtime state through `st.session_state`.
- I can point to where API key enforcement and runtime config overrides are implemented.